

AI Engineer Take-Home Project (4 Hours Max)

Note: Make any assumptions necessary to complete the project – and explain your assumptions.

Title: Utility Invoice Processing with LLMs

Objective

Build a lightweight system that extracts structured data from utility invoices (electricity, gas, or water) using an LLM, and outputs the results as a CSV file.

This exercise is designed to evaluate your ability to:

- Work with LLMs in a practical data extraction task
- Design simple but effective pipelines
- Handle multilingual inputs
- Make pragmatic engineering tradeoffs under time constraints

Project Requirements

Input

- A small set of sample utility invoices (PDFs or text-based documents)
- Include at least 3–5 invoices with variation in:
 - Format/layout
 - Language (at least 2 different languages, e.g., English + Spanish/French/etc.)

You may use publicly available examples or create mock invoices.

Output

Generate a CSV file with the following fields (at minimum):

- vendor_name
- invoice_date
- service_address (*if available*)
- utility_type (*electricity, gas, water*)
- usage_amount (*numeric*)
- usage_unit (*e.g., kWh, therms, gallons*)
- Billing_period_start
- billing_period_end

Core Task

Build a pipeline that:

1. Ingests invoice documents

2. Extracts text directly from the documents (assume text is already available; OCR is not required)
3. Uses an LLM to parse and structure the data
4. Handles multilingual content appropriately
5. Outputs a clean CSV

You may assume:

- Variation in invoice layouts
- Multiple languages
- Missing or ambiguous fields

Tools & Tech

You are encouraged to use modern “vibe coding” tools and AI-assisted environments such as:

- Replit
- Cursor
- VS Code + Copilot
- Claude / ChatGPT / other LLM APIs

You may use any libraries/frameworks you prefer (e.g., Python, Node.js, LangChain, etc.).

What We’re Looking For

We are not expecting production-grade systems. Instead, we care about:

- Clarity of approach
- Thoughtful use of LLMs (prompting, multilingual handling, structure, reliability)
- Handling of variability across formats and languages
- Simplicity and pragmatism
- Code readability

Bonus (optional):

- Language detection or normalization strategy
- Confidence scoring
- Basic validation checks
- Modular or extensible design

Deliverables

Please submit:

1. **Code Repository**

- Well-organized code
- Clear instructions to run the project
- Sample input files included

2. **Output CSV**

- Generated from your sample invoices

3. **Testing Approach**

Include a short section (in your README or as a separate document) that describes:

- How you validated the accuracy of extracted fields
- Any test cases or edge cases you considered (e.g., missing fields, different languages, unit variations)
- Whether you implemented any automated tests (optional, but encouraged)
- How you would improve testing if given more time

4. **Short Video (5–10 minutes)**

Walk us through:

- Your approach and key decisions
- System architecture
- How you handled multilingual inputs
- How you used LLMs
- Tradeoffs you made
- What you would improve with more time
- What you learned from the project

5. **Responses to the following questions:**

- Where are you located?
- Are you authorized to work in the country in which you reside?
- Will you now or in the future require sponsorship to work in your country of residence?
- When could you be available to start?
- What is your salary expectation? Please provide a range.
- This role requires daily collaboration with teams in both US Time and India Standard Time. Are you able to work the core collaboration hours required for this role? *Note: This can mean midnight if you're in US ET*
- Briefly describe your hands-on experience building or working with AI-applied products?
- Outline your experience across specifically the AI software development lifecycle? What was your role and what tools did you use for user stories, development, evals/testing, deployment, and maintenance?

Time Expectation

- Please spend no more than 4 hours on this project

- We value focus and prioritization over completeness

Evaluation Criteria

- Problem-solving approach
- Effective use of LLMs
- Handling of multilingual data
- Code quality and organization
- Testing and validation thinking
- Communication clarity (especially in the video)

Notes

- It's okay if parts are incomplete—explain your thinking
- If you make assumptions, document them
- Creativity is encouraged